

ASSIGNMENT 3 - BRIDGE AND ADAPTER

Topic: Photography Delivery System

Student: Aida Khalila Group: SE-2531

1. Problem and design goal

The system delivers individual photographs and photo albums using different delivery mechanisms. A single photo and an album have different preparation rules, while cloud, email, and a legacy photo service have different delivery behavior. Combining every photo type with every delivery mechanism in a separate subclass would duplicate logic and grow as more options are added. The application must also reuse an older delivery service without modifying that class.

2. Bridge: two independent dimensions

Abstraction: PhotoDelivery holds a DeliveryService reference. **Refined Abstractions:** SinglePhotoDelivery and AlbumDelivery implement their own send behavior. **Implementor:** DeliveryService declares deliver(String photoId, String recipient). **Concrete Implementors:** CloudDeliveryService, EmailDeliveryService, and LegacyDeliveryAdapter. AlbumDelivery adds an ALBUM- prefix before delegation. The photo type calls the interface rather than constructing or referencing a concrete delivery service.

This separates two axes of change. A new photo type can extend PhotoDelivery and use the existing DeliveryService contract; a new transport can implement DeliveryService and be supplied to the existing photo types. Thus the abstraction and implementation hierarchies can vary independently, supporting the Open/Closed Principle.

3. Adapter: genuine incompatibility

LegacyPhotoService does not implement DeliveryService. Its old operation is sendOldPhoto(...), not deliver(String, String). The legacy API uses a numeric photo identifier and a different argument contract; it also communicates failure through a status code, unlike the exception-based contract exposed by the new application. This is more than a simple method rename.

LegacyDeliveryAdapter implements DeliveryService and stores a LegacyPhotoService instance (object composition). It normalizes supported IDs such as OLD-001 and ALBUM-OLD-2026, converts the numeric part, delegates to the old method, and translates invalid identifiers into IllegalArgumentException and a reported legacy failure into IllegalStateException. The old class remains unchanged. PhotoDelivery sees only DeliveryService; it does not know legacy methods, status codes, or constants.

4. Why the patterns are combined

Bridge alone separates the two hierarchies but does not make the incompatible legacy API satisfy DeliveryService. Adapter alone makes the old service usable but does not separate photo-type variation from delivery-mechanism variation. The adapter therefore serves as one of the three Bridge implementors, solving both problems in one coherent design.

5. Chosen complexity module: Dynamic Implementor Selection

The chosen module is **Dynamic Implementor Selection**, not a two-way adapter. `DeliveryServiceSelector.select(deliveryType)` maps a runtime input string to cloud, email, or the adapted legacy implementation. Main reads `args[0]` when an argument is provided and otherwise uses legacy as a demonstration default. The selected service is passed into `AlbumDelivery` through its constructor. The selection logic is centralized rather than spread through the refined abstractions.

The demonstration also constructs cloud and email examples directly to show the Bridge combinations; the selected-delivery path uses runtime input. Adding another implementation does not require changes to `PhotoDelivery`, `SinglePhotoDelivery`, or `AlbumDelivery`, although the current selector mapping must be extended.

6. Test strategy and observed results

JUnit 5 checks ordinary delivery and the selector behavior. `BridgeAdapterTest` passed seven tests, including invalid identifier handling, legacy error translation, and both stub-based delegation checks. `DeliveryServiceSelectorTest` passed four tests, for eleven passing tests in total. Main produced cloud, email, and two legacy delivery messages and finished with exit code 0.

For delegation tests, a stub `DeliveryService` records the received photo ID and recipient. A `SinglePhotoDelivery` test checks that the original ID reaches the stub; an `AlbumDelivery` test checks that ALBUM- is prepended before delegation. For failure translation, tests expect an application-level exception rather than exposing a legacy-specific result. The standard test command is **mvn test**.

7. SOLID, extension examples, and limitation

OCP: a new refined abstraction (for example, a new photo-delivery format) can reuse the existing implementors; a new delivery implementation can implement `DeliveryService` without editing the abstraction classes. **SRP:** photo-format behavior, delivery behavior, legacy conversion, and runtime selection have separate responsibilities. **Dependency inversion:** `PhotoDelivery` depends on `DeliveryService` rather than concrete delivery classes.

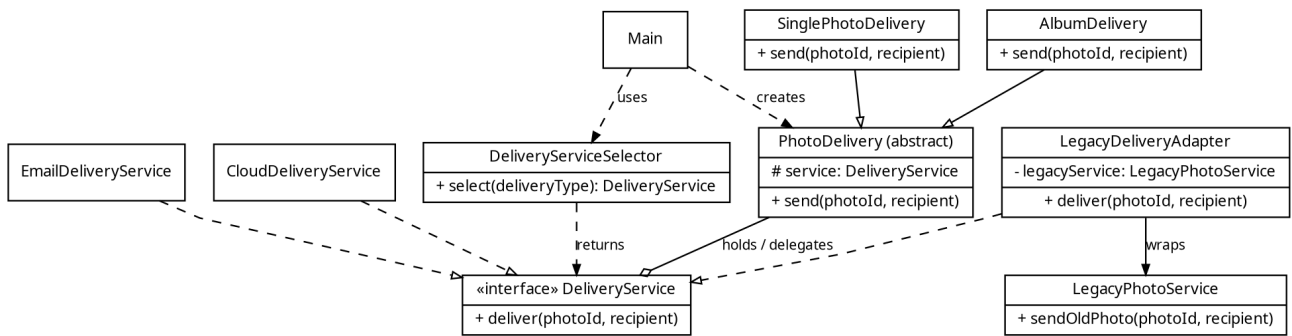
Limitation: legacy identifiers must follow the supported formats; unsupported strings cannot be converted into the legacy numeric identifier. In addition, adding a new service requires a new mapping in `DeliveryServiceSelector`. The runtime default is useful for demonstration but should be replaced by explicit input validation in a production command-line interface.

8. Conclusion

The design combines Bridge and Adapter for one practical integration problem: photo types and delivery services evolve independently, while an unchanged legacy service participates through the same implementor contract. The UML diagram on the following page documents the project-specific class relationships.

APPENDIX A - UML CLASS DIAGRAM

Project-specific diagram (separate from the two-page design rationale)



Bridge: PhotoDelivery delegates to DeliveryService. Adapter: LegacyDeliveryAdapter implements DeliveryService and wraps LegacyPhotoService.